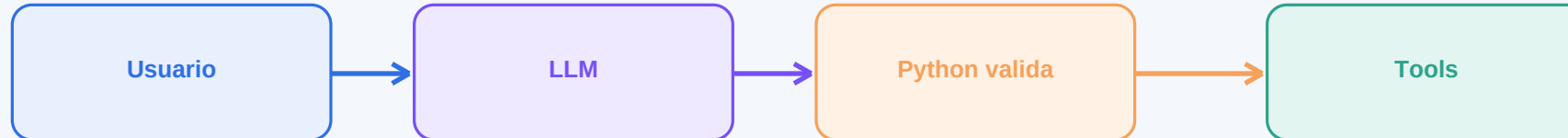


Nivel Avanzado

Tool Calling y Agentes Multipaso

Del asistente con memoria a un agente que solicita herramientas, valida resultados y encadena varios pasos.



BÁSICO · COMPLETADO

INTERMEDIO · COMPLETADO

AVANZADO · COMPLETADO

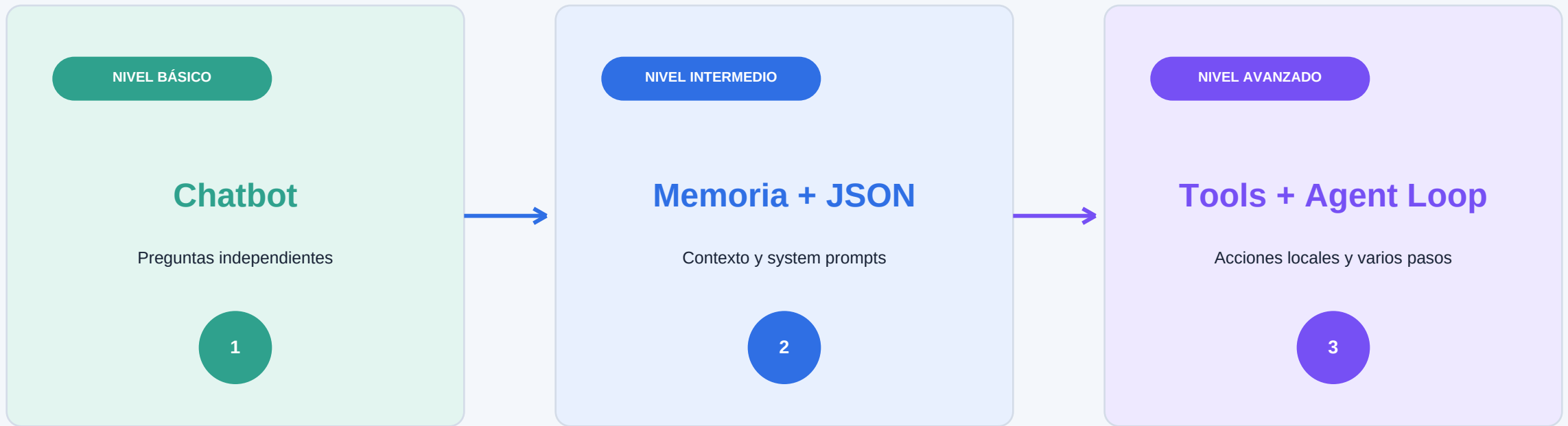
EXPERTO · PENDIENTE

Dos herramientas locales · validación estricta · bucle agentic · límite de seguridad



1. Evolución del proyecto

Cada nivel conserva lo anterior y añade una capacidad nueva.

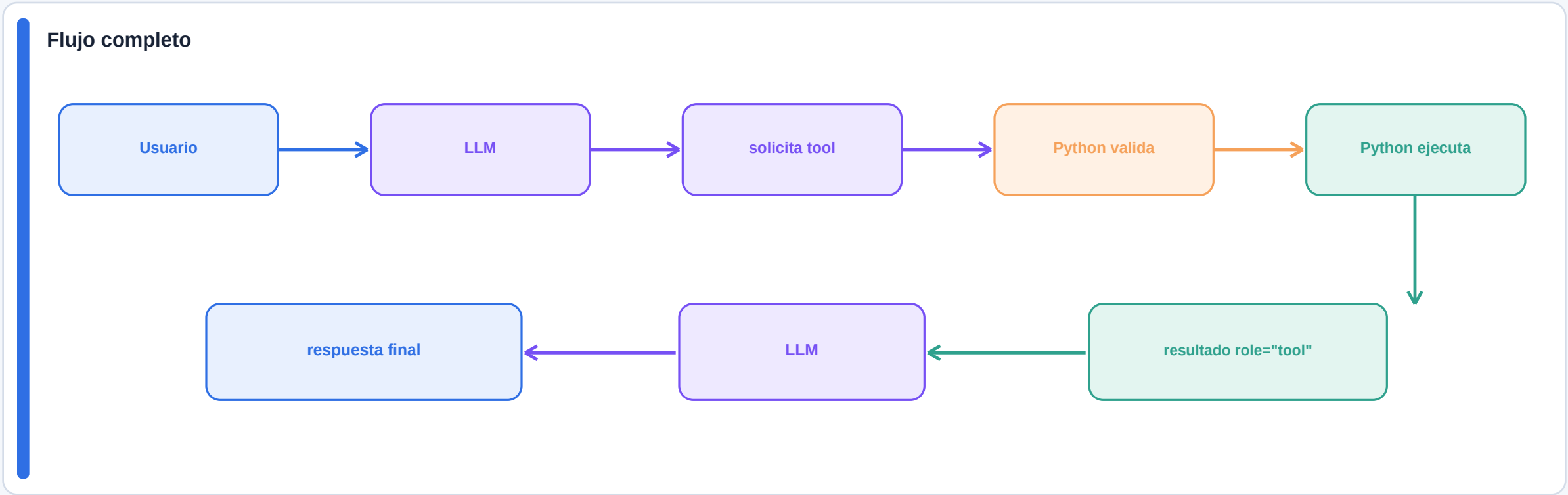


Qué añade el Nivel Avanzado

El modelo puede solicitar herramientas, recibir sus resultados con `role="tool"` y continuar hasta construir una respuesta final.

2. Qué es Tool / Function Calling

El modelo describe la acción; Python mantiene el control real de la ejecución.



EL MODELO NO EJECUTA PYTHON DIRECTAMENTE

El LLM solicita. Python valida, decide y ejecuta.

3. Herramienta calculate

Una calculadora local con operaciones cerradas y argumentos validados.

calculate(operation, a, b)

add

subtract

multiply

divide

- JSON Schema define los tres argumentos.
- operation usa un enum cerrado.
- a y b deben ser números finitos.
- divide rechaza la división entre cero.
- No existe eval().

Ejemplo local

25 × 6



calculate



150



RESULTADO COMPROBADO: 150

Seguridad: el nombre de la herramienta, la operación, la estructura JSON, los tipos y los valores se comprueban antes de calcular.

4. Herramienta get_task_info

Consulta prioridad, duración y categoría en un archivo JSON local.

get_task_info(task_name)

```
TASKS_FILE = data/tareas.json
{"task_name": "estudiar Python"}
-> prioridad, duración y categoría
```

- task_name debe ser texto no vacío.
- La búsqueda ignora mayúsculas y espacios exteriores.
- Los datos son ficticios, locales y no sensibles.

estudiar Python

prioridad: alta
duración: 90 minutos
categoría: estudio

preparar presentación

prioridad: media
duración: 60 minutos
categoría: trabajo

ir al gimnasio

prioridad: media
duración: 75 minutos
categoría: salud

hacer compra

prioridad: alta
duración: 45 minutos
categoría: hogar

Fuente data/tareas.json

Estructura validada antes de devolver la tarea al modelo.

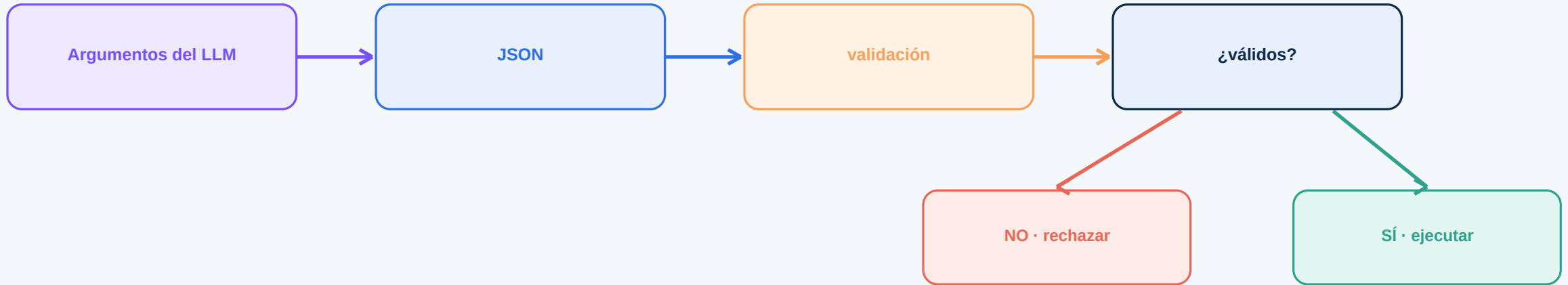
5. Selección entre herramientas

El modelo decide qué capacidad necesita; Python limita el catálogo permitido.



6. JSON Schema y validación

Los argumentos generados por el modelo nunca se ejecutan sin comprobarlos.



Ejemplos rechazados antes de ejecutar

HERRAMIENTA DESCONOCIDA

JSON INVÁLIDO

OPERATION NO PERMITIDA

TIPOS INCORRECTOS

TASK_NAME AUSENTE

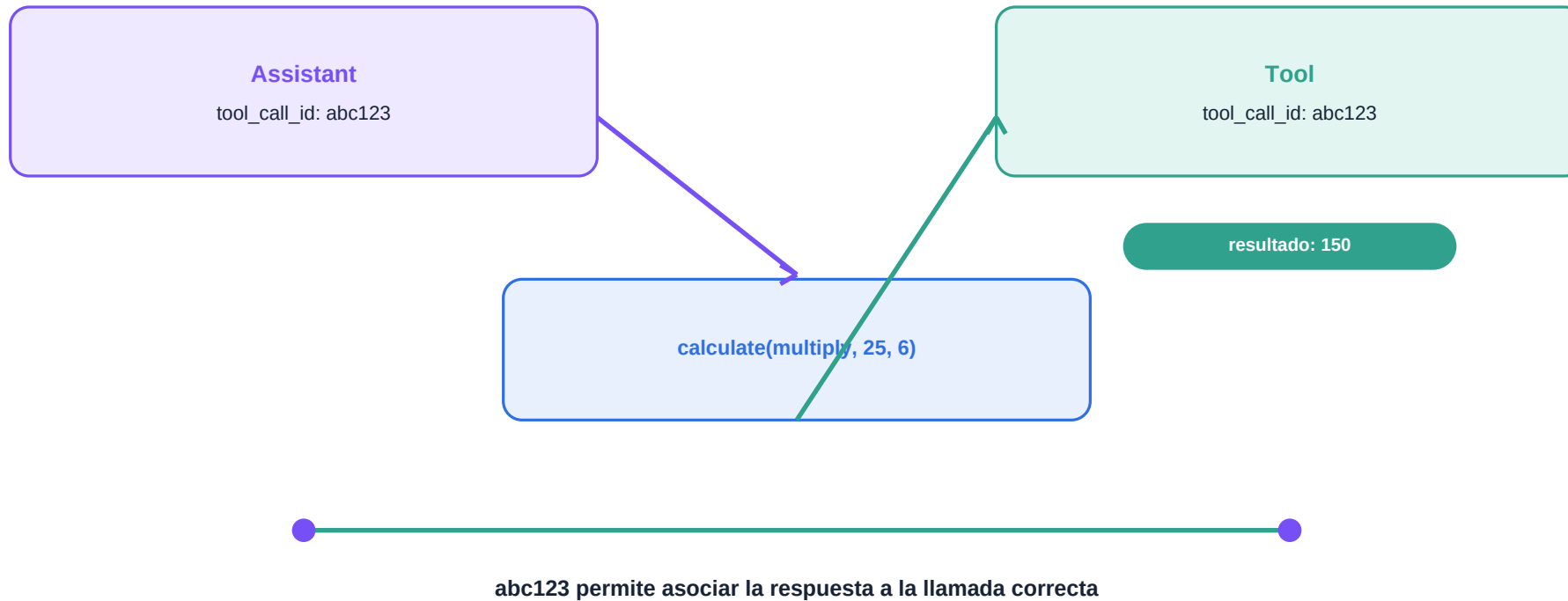
ARGUMENTOS ADICIONALES

NÚMEROS NO FINITOS

7. tool_call_id y role="tool"

El identificador enlaza cada resultado con la solicitud exacta que lo originó.

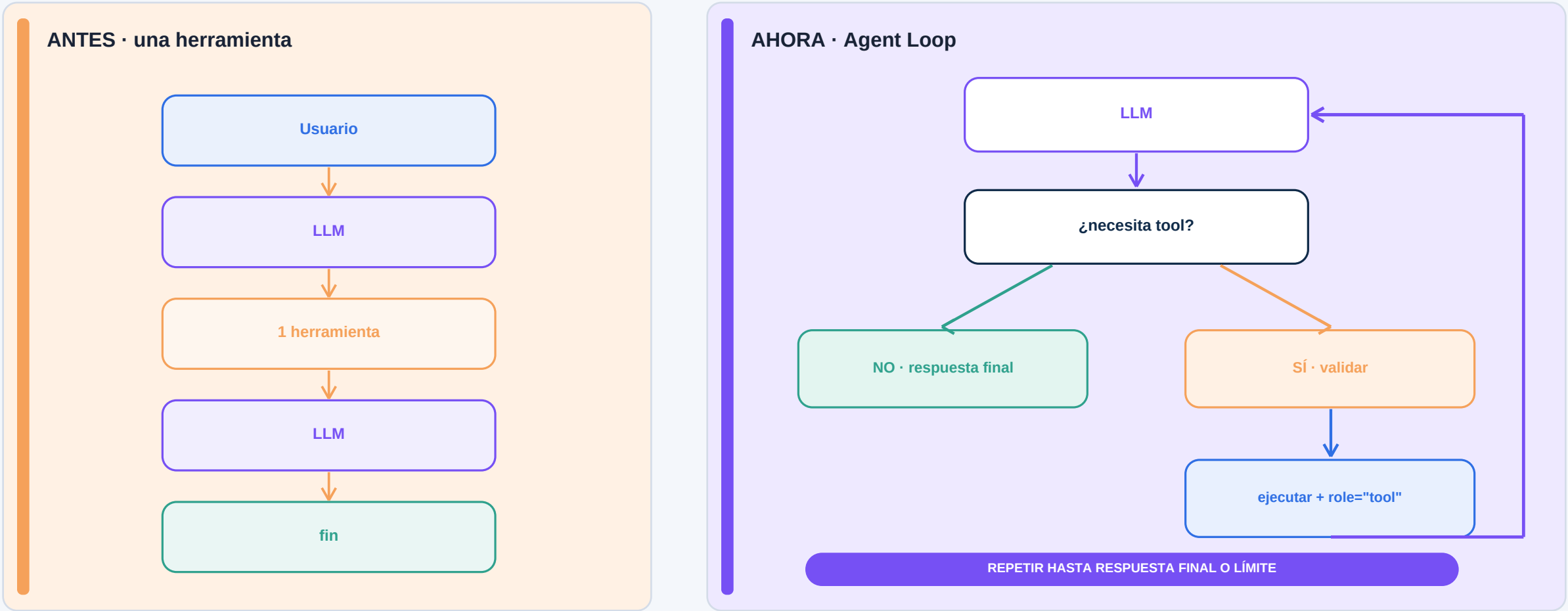
Una solicitud y su resultado comparten el mismo ID



El resultado vuelve al historial con role="tool" antes de consultar otra vez al LLM.

8. Evolución al bucle agentic

El flujo deja de terminar después de una herramienta y puede repetirse hasta obtener una respuesta final.

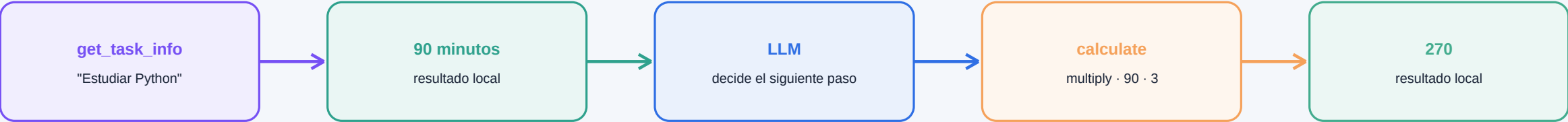


9. Prueba multipaso real

Una consulta necesita primero datos locales y después una operación matemática.

USUARIO

¿Cuánto dura estudiar Python y cuántos minutos serían si hiciera esa tarea 3 veces?



RESPUESTA FINAL

"Estudiar Python tiene una duración de 90 minutos. Si lo haces 3 veces serían 270 minutos."

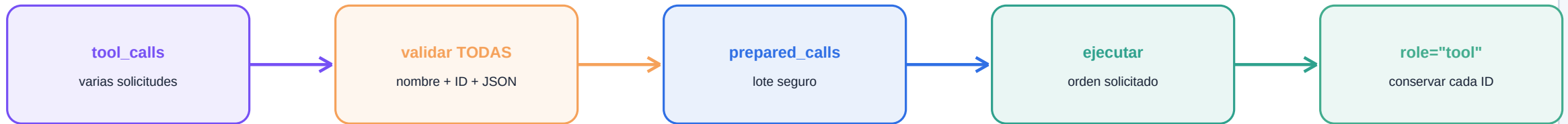
EL ORDEN get_task_info → calculate FUE DECIDIDO POR EL MODELO

El encadenado no estaba programado manualmente como una secuencia fija.

10. Múltiples tool_calls

Una misma respuesta puede solicitar varias herramientas; el código prepara el lote antes de ejecutar.

Validar todas primero, ejecutar después



- Se rechaza el lote completo si una solicitud no es válida.
- Ninguna función se ejecuta mientras queden llamadas sin validar.
- Los resultados se añaden en el mismo orden pedido por el modelo.

Por qué importa

Validar el lote completo evita ejecuciones parciales: no se realiza la primera acción para descubrir después que la segunda era insegura o incoherente.

11. MAX_AGENT_STEPS

Un límite configurable impide que el agente solicite herramientas indefinidamente.

MAX_AGENT_STEPS = 5



"Error: se alcanzó el máximo de pasos del agente sin una respuesta final."

El límite se comprobó localmente con un cliente simulado. El programa termina de forma controlada y sin traceback.

12. Seguridad del agente

El modelo propone acciones dentro de un perímetro explícito; nunca obtiene ejecución arbitraria.



13. Pruebas realizadas

Rutas principales y casos límite terminaron de forma controlada.

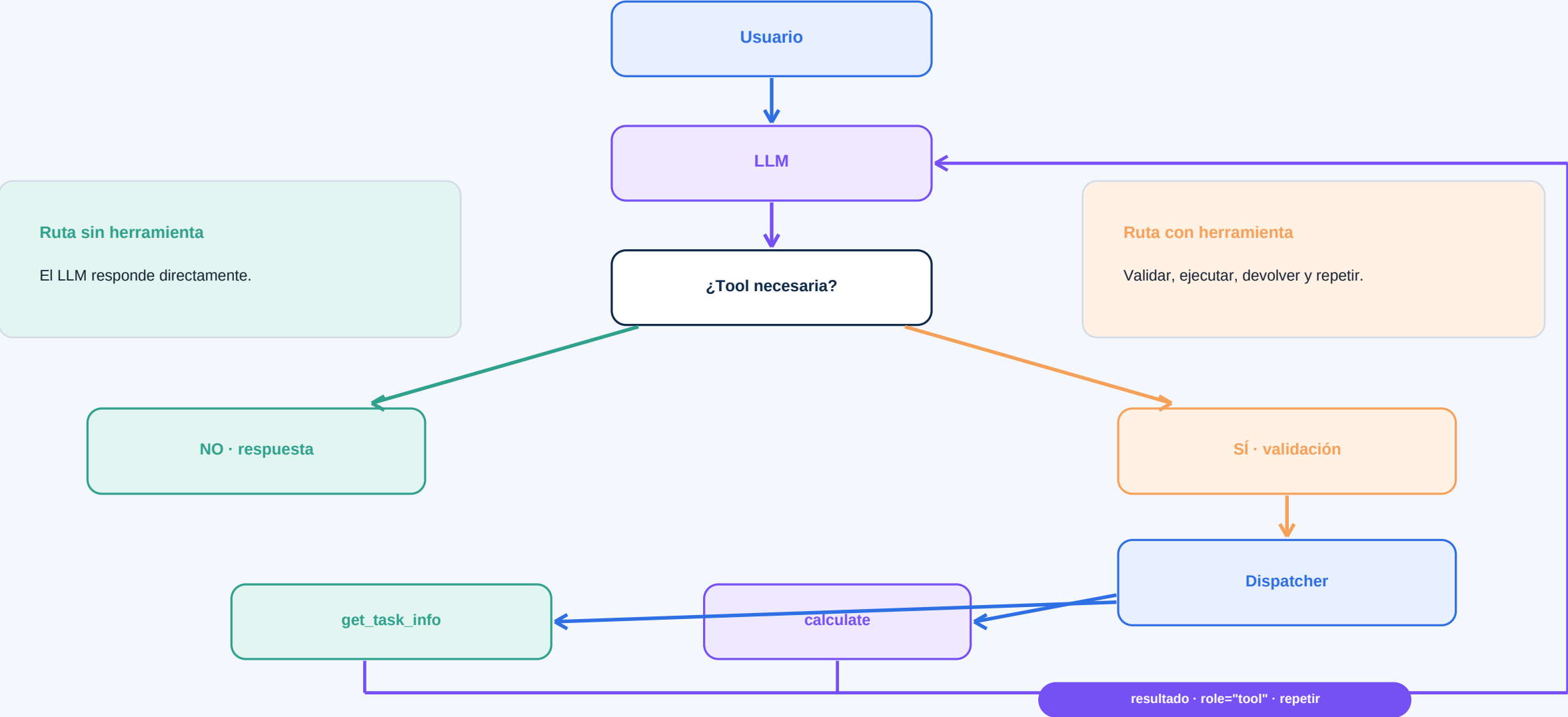
PRUEBA	RUTA	RESULTADO	ESTADO
MULTIPASO	get_task_info → calculate	270 minutos	PASS
CALCULADORA	calculate · 25 × 6	150	PASS
TAREA	get_task_info · hacer compra	prioridad alta	PASS
SIN TOOL	pregunta general	respuesta directa	PASS

Casos límite controlados sin traceback

- HERRAMIENTA DESCONOCIDA
- ARGUMENTOS INVÁLIDOS
- DIVISIÓN ENTRE CERO
- TAREA INEXISTENTE
- RESPUESTA VACÍA
- MAX_AGENT_STEPS

14. Arquitectura final del Nivel Avanzado

Un circuito cerrado entre el LLM, la validación y dos herramientas locales.



15. Commits del Nivel Avanzado

La evolución real separa cada capacidad en un cambio trazable.

Línea temporal



Cada commit añade una capacidad concreta: calculadora, consulta local, bucle multipaso y cierre formal del nivel.

16. Conceptos aprendidos

Las piezas que convierten un chatbot con memoria en un agente controlado.

Mapa de conceptos



Estado del reto

Básico	100%	COMPLETADO
Intermedio	100%	COMPLETADO
Avanzado	100%	COMPLETADO
Experto	0%	PENDIENTE

Diferencia frente al Intermedio

Ahora el modelo también puede solicitar acciones locales y continuar el razonamiento.



El contenido del Nivel Experto queda pendiente y no se desarrolla en este documento.